CHAPTER 3

STACKS AND QUEUES

All the programs in this file are selected from

Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed

“Fundamentals of Data Structures in C”,

Stack (堆疊) : a Last-In-First-Out (LIFO) list (後進先出串列)

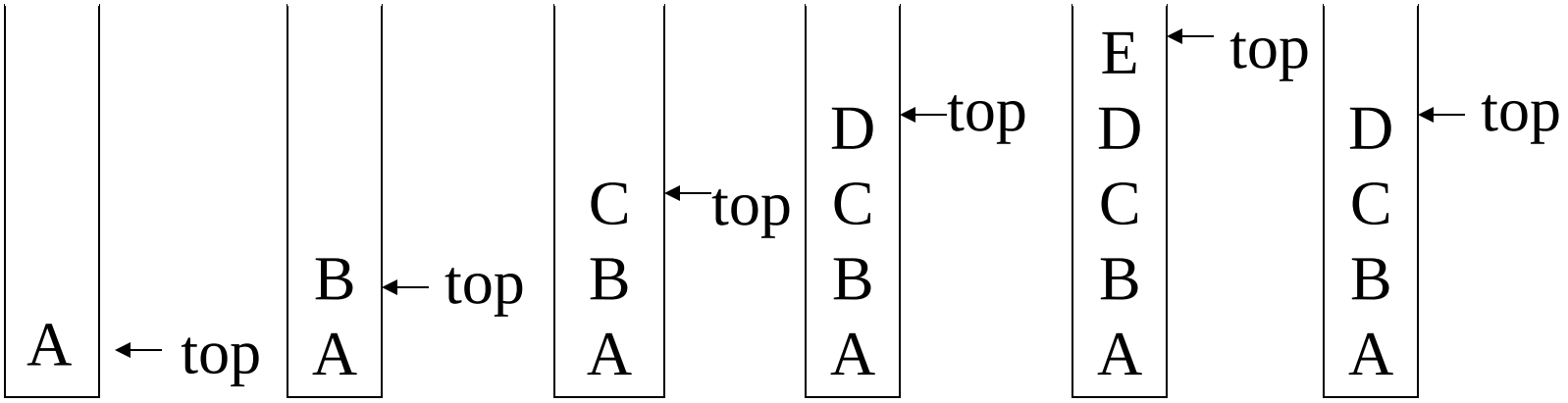


***Figure 3.1:** Inserting (推入) and deleting (彈出) elements in a stack (p.108)

an application of stack: stack frame (堆疊框) of function call (函式呼叫)

(activation record, 活動紀錄)

fp: a pointer to current stack frame

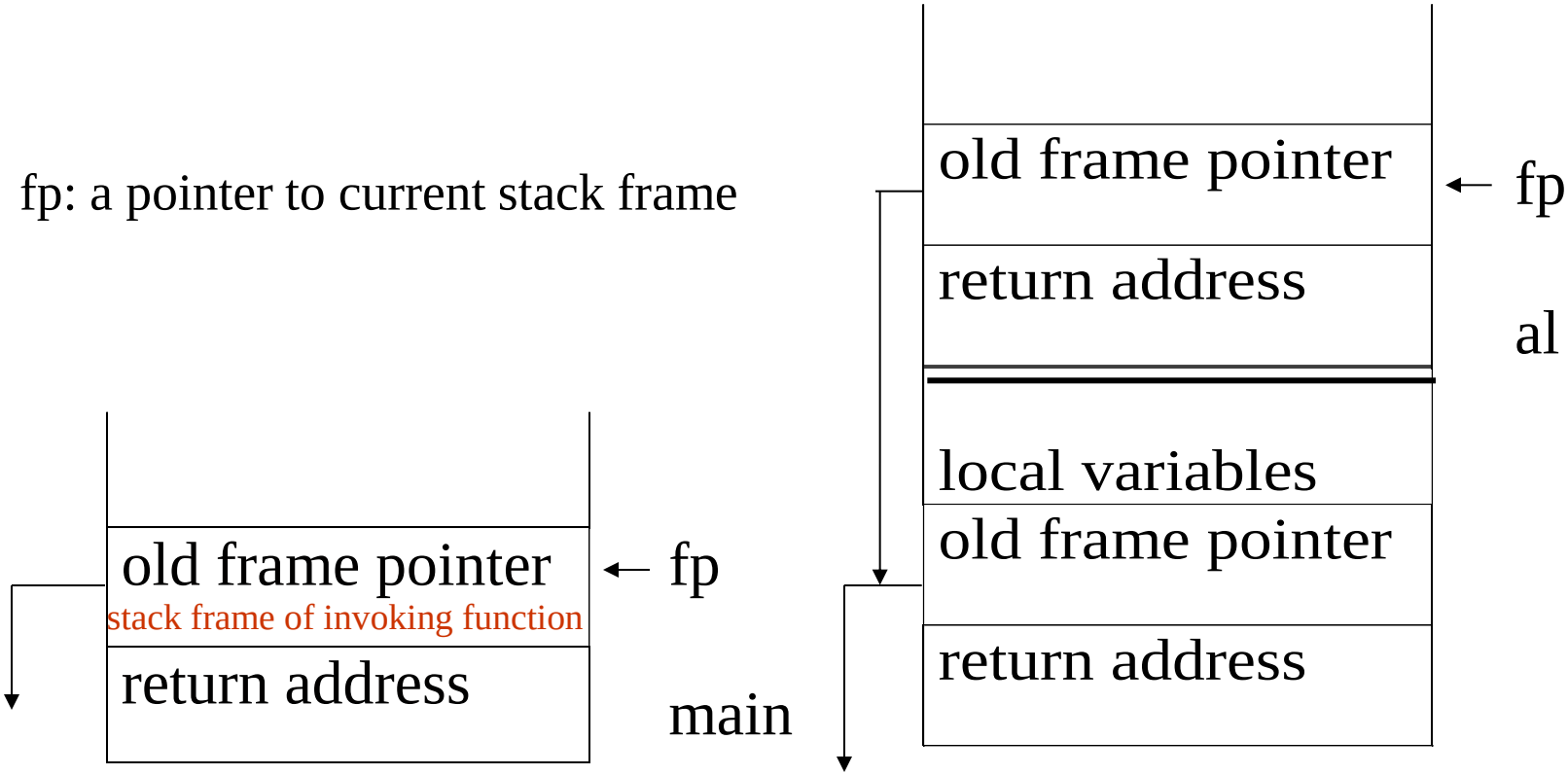


***Figure 3.2:** System stack after function call a1 (p.109)

abstract data type for stack

structure *Stack* is

objects: a finite ordered list with zero or more elements. (含有零個或多個元素的有限有序串列)

functions:

for all *stack* ∈ *Stack*, *item* ∈ *element*, *max_stack_size* ∈ positive integer

Stack CreateS(*max_stack_size*) ::=

create an empty stack whose maximum size is *max_stack_size*

Boolean IsFull(*stack*, *max_stack_size*) ::=

if (number of elements in *stack* == *max_stack_size*)

return TRUE

else return FALSE

Stack Add(*stack*, *item*) ::=

if (IsFull(*stack*)) *stack_full*

else insert *item* into top of *stack* and **return**

Boolean IsEmpty(*stack*) ::=
if(*stack* == CreateS(*max_stack_size*))
return TRUE
else return FALSE

Element Delete(*stack*) ::=
if(IsEmpty(*stack*)) **return**
else remove and return the *item* on the top of the stack.

***Structure 3.1:** Abstract data type *Stack* (p.110)

Implementation: using array

***Stack* CreateS(max_stack_size)** ::=

```
#define MAX_STACK_SIZE 100 /* maximum stack size */
typedef struct {
        int key;
        /* other fields */
        } element;
element stack[MAX_STACK_SIZE];
int top = -1;
```

***Boolean* IsEmpty(Stack)** ::= top< 0;

***Boolean* IsFull(Stack)** ::= top >= MAX_STACK_SIZE-1;

Add to a stack

```
void add(int *top, element item)
{
 /* add an item to the global stack */
    if (*top >= MAX_STACK_SIZE-1)  {
         stack_full( );
         return;
    }
    stack[++*top] = item;
}
```

***Program 3.1:** Add to a stack (p.111)

Delete from a stack

```
element delete(int *top)
{
 /* return the top element from the stack */
    if (*top == -1)
        return stack_empty( );  /* returns and error key */
    return stack[(*top)--];
 }
```

***Program 3.2:** Delete from a stack (p.111)

Queue (佇列): a First-In-First-Out (FIFO) list (先進先出串列)

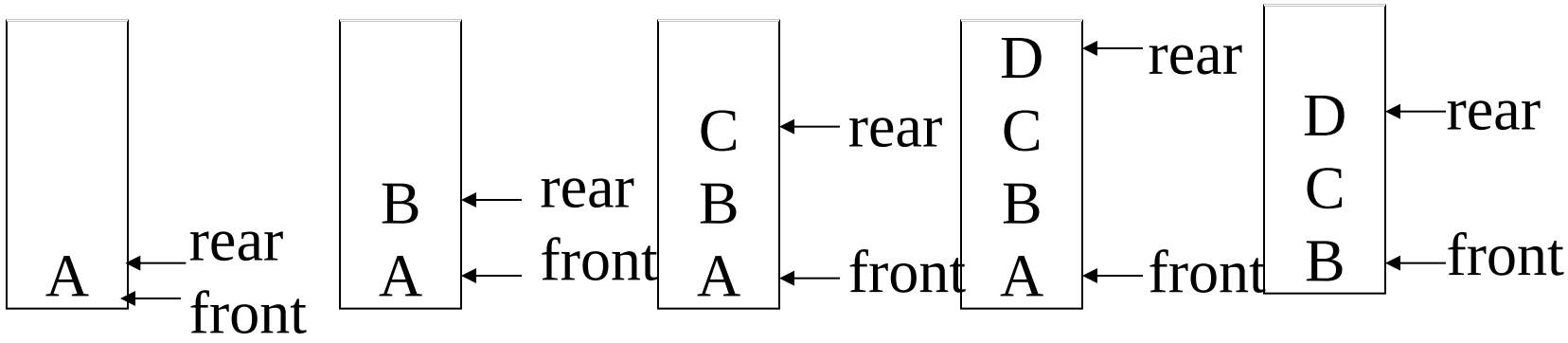


***Figure 3.4:** Inserting (插入) and deleting (刪除) elements in a queue (p.114)

Application: Job scheduling (工作排程)

front	rear	Q[0]	Q[1]	Q[2]	Q[3]	Comments
-1	-1					queue is empty
-1	0	J1				Job 1 is added
-1	1	J1	J2			Job 2 is added
-1	2	J1	J2	J3		Job 3 is added
0	2		J2	J3		Job 1 is deleted
1	2			J3		Job 2 is deleted

***Figure 3.5:** Insertion and deletion from a sequential queue (p.117)

Abstract data type of queue

structure *Queue* is

objects: a finite ordered list with zero or more elements. (含有零個或多個元素的有限有序串列)

functions:

for all *queue* ∈ *Queue*, *item* ∈ *element*,

max_ queue_ size ∈ positive integer

Queue CreateQ(*max_queue_size*) ::=

create an empty queue whose maximum size is *max_queue_size*

Boolean IsFullQ(*queue*, *max_queue_size*) ::=

if(number of elements in *queue* == *max_queue_size*)

return *TRUE*

else return *FALSE*

Queue AddQ(*queue*, *item*) ::=

if (IsFullQ(*queue*)) *queue_full*

else insert *item* at rear of *queue* and return *queue*

Boolean IsEmptyQ(*queue*) ::=
    if (*queue* ==CreateQ(*max_queue_size*))
    return *TRUE*
    else return *FALSE*
Element DeleteQ(*queue*) ::=
    if (IsEmptyQ(*queue*)) **return**
    else remove and return the *item* at front of queue.

***Structure 3.2:** Abstract data type *Queue (p.115)*

Implementation 1: using array

Queue CreateQ(*max_queue_size*) ::=

```
# define MAX_QUEUE_SIZE 100/* Maximum queue size */
typedef struct {
            int key;
            /* other fields */
            } element;
element queue[MAX_QUEUE_SIZE];
int rear = -1;
int front = -1;
```

Boolean IsEmpty(queue) ::= front == rear

Boolean IsFullQ(queue) ::= rear == MAX_QUEUE_SIZE-1

Add to a queue

```
void addq(int *rear, element item)
{
/* add an item to the queue */
    if (*rear == MAX_QUEUE_SIZE_1) {
       queue_full( );
       return;
    }
   queue [++*rear] = item;
}
```

***Program 3.5:** Add to a queue (p.116)

Delete from a queue

```
element deleteq(int *front, int rear)
{
/* remove element at the front of the queue */
    if ( *front == rear)
        return queue_empty( );     /* return an error key */
    return queue [++ *front];
}
```

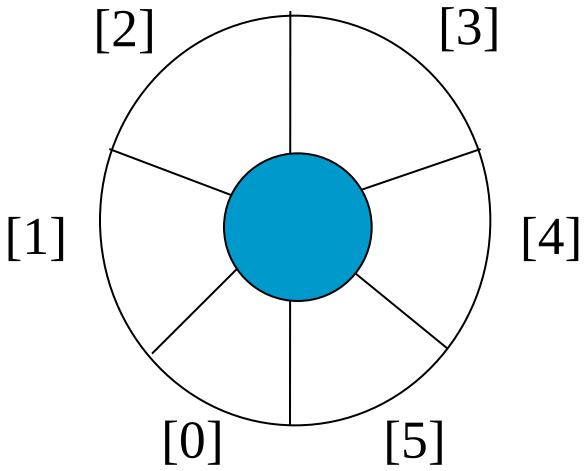
***Program 3.6:** Delete from a queue(p.116)

Implementation 2: regard an array as a circular queue (環狀佇列)

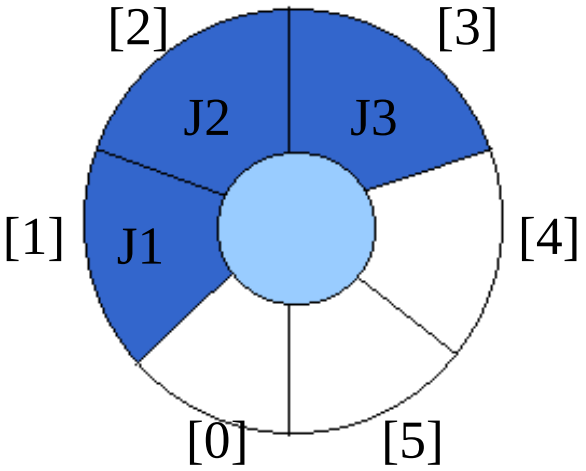
front: one position counterclockwise (逆時針方向) from the first element

rear: current end

EMPTY QUEUE



front = 0
rear = 0

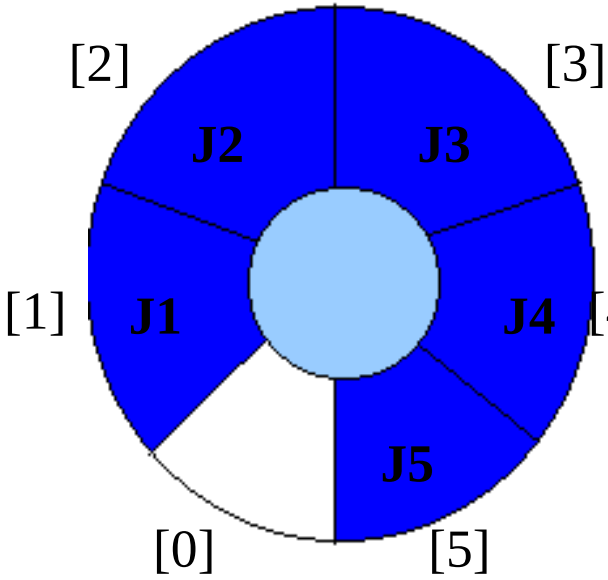


front = 0
rear = 3

***Figure 3.6:** Empty and nonempty circular queues (p.117)

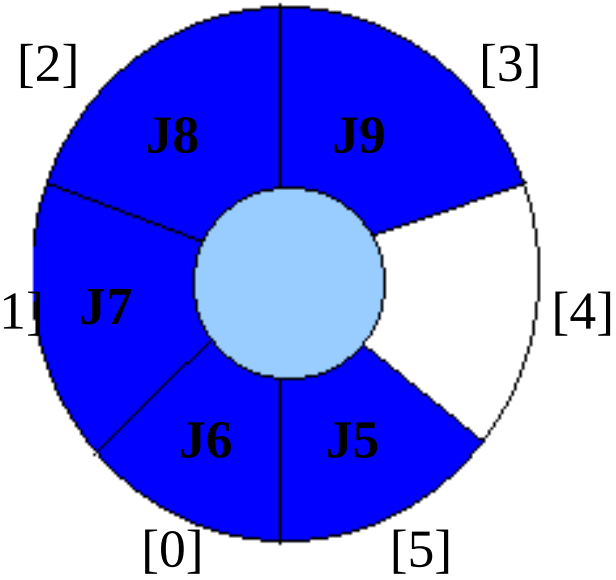
Problem: one space is left when queue is full

FULL QUEUE



front =0
rear = 5

FULL QUEUE



front =4
rear =3

Add to a circular queue

```
void addq(int front, int *rear, element item)
{
/* add an item to the queue */
    *rear = (*rear +1) % MAX_QUEUE_SIZE;
     if (front == *rear) /* reset rear and print error */
     return;
   }
     queue[*rear] = item;
}
```

***Program 3.7:** Add to a circular queue (p.118)

Delete from a circular queue

```
element deleteq(int* front, int rear)
{
   element item;
   /* remove front element from the queue and put it in item */
      if (*front == rear)
         return queue_empty( );
                  /* queue_empty returns an error key */
      *front = (*front+1) % MAX_QUEUE_SIZE;
      return queue[*front];
}
```

***Program 3.8:** Delete from a circular queue (p.119)

Evaluation of Expressions (運算式的計算)

X = a / b - c + d * e - a * c

a = 4, b = c = 2, d = e = 3

Interpretation 1:
((4/2)-2)+(3*3)-(4*2)=0 + 8+9=1

Interpretation 2:
(4/(2-2+3))*(3-4)*2=(4/3)*(-1)*2=-2.66666…

How to generate the machine instructions corresponding to a given expression?

precedence rule (優先權) + associative rule (結合性)

Token	Operator	Precedence[1]	Associativity
( ) [ ] -> .	function call array element struct or union member	17	left-to-right
-- ++	increment, decrement[2]	16	left-to-right
-- ++ ! - - + & * sizeof	decrement, increment[3] logical not one's complement unary minus or plus address or indirection size (in bytes)	15	right-to-left
(type)	type cast	14	right-to-left
* / %	mutiplicative	13	Left-to-right

+ -	binary add or subtract	12	left-to-right
<< >>	shift	11	left-to-right
> >= < <=	relational	10	left-to-right
== !=	equality	9	left-to-right
&	bitwise and	8	left-to-right
^	bitwise exclusive or	7	left-to-right
|	bitwise or	6	left-to-right
&&	logical and	5	left-to-right
||	logical or	4	left-to-right

?:	conditional	3	right-to-left
= += -= /= *= %= <<= >>= &= ^= |=	assignment	2	right-to-left
,	comma	1	left-to-right

1.The precedence column is taken from Harbison and Steele.
2.Postfix form (後置式)

***Figure 3.12:** Precedence hierarchy (優先權順序) for C (p.130)

user **compiler**

Infix	Postfix
2+3*4	234*+
a*b+5	ab*5+
(1+2)*7	12+7*
a*b/c	ab*c/
(a/(b-c+d))*(e-a)*c	abc-d+/ea-*c*
a/b-c+d*e-a*c	**ab/c-de*+ac*-**

***Figure 3.13:** Infix and postfix notation (中置和後置的表示法) (p.131)

Postfix: no parentheses (沒有括號), no precedence

Token	Stack [0]	[1]	[2]	Top
6	6			0
2	6	2		1
/	6/2			0
3	6/2	3		1
-	6/2-3			0
4	6/2-3	4		1
2	6/2-3	4	2	2
*	6/2-3	4*2		1
+	6/2-3+4*2			0

***Figure 3.14:** Postfix evaluation (後置計算) (p.131)

Goal: infix --> postfix (中置轉後置)

Assumptions:

operators: +, -, *, /, %

operands: single digit integer

```
#define MAX_STACK_SIZE 100 /* maximum stack size */
#define MAX_EXPR_SIZE 100 /* max size of expression */
typedef enum{1paran, rparen, plus, minus, times, divide,
                    mod, eos, operand} precedence;
int stack[MAX_STACK_SIZE]; /* global stack */
char expr[MAX_EXPR_SIZE]; /* input string */
```

```
int eval(void)
{
/* evaluate a postfix expression, expr, maintained as a
   global variable, '\0' is the the end of the expression.
   The stack and top of the stack are global variables.
   get_token is used to return the token type and
   the character symbol. Operands are assumed to be single
   character digits */
  precedence token;
  char symbol;
  int op1, op2;
  int n = 0;  /* counter for the expression string */
  int top = -1;
  token = get_token(&symbol, &n);
  while (token != eos)  {
     if (token == operand)          exp: character array
         add(&top, symbol-'0');   /* stack insert */
```

```
    else {
        /* remove two operands, perform operation, and
           return result to the stack */
      op2 = delete(&top);  /* stack delete */
      op1 = delete(&top);
      switch(token) {
          case plus: add(&top, op1+op2); break;
          case minus: add(&top, op1-op2); break;
          case times: add(&top, op1*op2); break;
          case divide: add(&top, op1/op2); break;
          case mod: add(&top, op1%op2);
      }
    }
    token = get_token (&symbol, &n);
  }
  return delete(&top); /* return result */
}
```

***Program 3.13:** Function to evaluate a postfix expression (計算後置運算式的函式) (p.133)

```
precedence get_token(char *symbol, int *n)
{
/* get the next token, symbol is the character
   representation, which is returned, the token is
   represented by its enumerated value, which
   is returned in the function name */

  *symbol =expr[(*n)++];
  switch (*symbol)  {
     case '(' : return lparen;
     case ')' : return rparen;
     case '+': return plus;
     case '-' : return minus;
```

```
    case '/' :  return divide;
    case '*' : return times;
    case '%' : return mod;
    case '\0' : return eos;
    default  : return operand;
                /* no error checking, default is operand */
    }
}
```

***Program 3.14:** Function to get a token from the input string (從輸入字串取得標記的函式) (p.134)

Infix to Postfix Conversion
(Intuitive Algorithm)

(1) Fully parenthesize expression

a / b - c + d * e - a * c -->

((((a / b) - c) + (d * e)) - a * c))

(2) All operators replace their corresponding right parentheses.

((((a / b) - c) + (d * e)) - a * c))



(3) Delete all parentheses.

ab/c-de*+ac*-

two passes

The orders of operands in infix and postfix are the same.

a + b * c, * > +

Token	Stack [0] [1] [2]	Top	Output
a		-1	a
+	+	0	a
b	+	0	ab
*	+ *	1	ab
c	+ *	1	abc
eos		-1	abc* +

***Figure 3.15:** Translation of a+b*c to postfix (p.135)

a $*_1$ (b +c) $*_2$ d

Token	Stack [0]	[1]	[2]	Top	Output
a				-1	a
$*_1$	$*_1$			0	a
(	$*_1$	(		1	a
b	$*_1$	(		1	ab
+	$*_1$	(	+	2	ab
c	$*_1$	(	+	2	abc
)	$*_1$	match)		0	abc+
$*_2$	$*_2$	$*_1 = *_2$		0	abc+$*_1$
d	$*_2$			0	abc+$*_1$d
eos	$*_2$			0	abc+$*_1$d$*_2$

*** Figure 3.16:** Translation of a*(b+c)*d to postfix (p.135)

Rules

(1) Operators are taken out of the stack as long as their in-stack precedence (堆疊內優先權) is higher than or equal to the incoming precedence (進堆疊優先權) of the new operator.

(2) (has low in-stack precedence, and high incoming precedence.

	(	)	+	-	*	/	%	eos
isp	0	19	12	12	13	13	13	0
icp	20	19	12	12	13	13	13	0

```
precedence stack[MAX_STACK_SIZE];
/* isp and icp arrays -- index is value of precedence
lparen, rparen, plus, minus, times, divide, mod, eos */
static int isp [ ] = {0, 19, 12, 12, 13, 13, 13, 0};
static int icp [ ] = {20, 19, 12, 12, 13, 13, 13, 0};
```

isp: in-stack precedence

icp: incoming precedence

```
void postfix(void)
{
/* output the postfix of the expression. The expression
    string, the stack, and top are global */
   char symbol;
   precedence token;
   int n = 0;
   int top = 0; /* place eos on stack */
   stack[0] = eos;
   for (token = get _token(&symbol, &n); token != eos;
                    token = get_token(&symbol, &n)) {
     if (token == operand)
        printf (“%c”, symbol);
     else if (token == rparen ){
```

```
    /*unstack tokens until left parenthesis */
    while (stack[top] != lparen)
        print_token(delete(&top));
    delete(&top); /*discard the left parenthesis */
   }
   else{
    /* remove and print symbols whose isp is greater
        than or equal to the current token's icp */
    while(isp[stack[top]] >= icp[token] )
        print_token(delete(&top));
    add(&top, token);
   }
  }
  while ((token = delete(&top)) != eos)
      print_token(token);
  print("\n");
}
```

$\theta(n)$

$f(n)=\theta(g(n))$ iff there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \le f(n) \le c_2 g(n)$ for all n, $n \ge n_0$.

$f(n)=\theta(g(n))$ iff $g(n)$ is both an upper and lower bound on $f(n)$.

***Program 3.15**: Function to convert from infix to postfix (中置轉成後置的函式) (p.137)